High-speed Autonomous Drifting with Deep Reinforcement Learning

Peide Cai*[1], Xiaodong Mei*[1], Lei Tai[2], Yuxiang Sun[1], and Ming Liu[1]

arXiv:2001.01377v1 [cs.RO] 6 Jan 2020

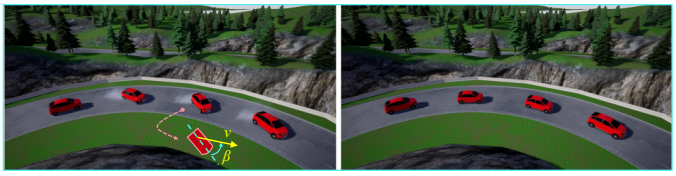
***Abstract*—Drifting is a complicated task for autonomous vehicle control. Most traditional methods in this area are based on motion equations derived by the understanding of vehicle dynamics, which is difficult to be modeled precisely. We propose a robust drift controller without explicit motion equations, which is based on the latest model-free deep reinforcement learning algorithm soft actor-critic. The drift control problem is formulated as a trajectory following task, where the error-based state and reward are designed. After being trained on tracks with different levels of difficulty, our controller is capable of making the vehicle drift through various sharp corners quickly and stably in the unseen map. The proposed controller is further shown to have excellent generalization ability, which can directly handle unseen vehicle types with different physical properties, such as mass, tire friction, etc.**

***Index Terms*—Deep reinforcement learning, deep learning in robotics and automation, racing car, motion control, field robots.**

I. Introduction

In motorsport of rallying, high-speed sideslip cornering, known as drifting, represents an attractive vehicle control maneuver undertaken by professional racing drivers. The slip angle β is measured by the angle between the direction of the heading (longitudinal axis of the vehicle) and the direction of the velocity vector at the centre of gravity, as shown in Fig. 1(a). In order to make a quick turn through sharp corners, skilled drivers execute drifts by deliberately inducing deep saturation of the rear tires by oversteering [1] or using the throttle [2], thereby destabilising the vehicle. They then stabilise the vehicle as it begins to spin by controlling it under a high sideslip configuration (up to 40 degrees [3]). Vehicle instability and corresponding control difficulty both increase as the sideslip angle increases. Therefore, drifting is a challenging control technique to operate the vehicle efficiently and safely beyond its handling limits. Compared with the normal cornering in which slipping is usually avoided by lowering the speed and making gentle turns (Fig. 1(b)), high-speed drifting techniques can help reduce the lap time during racing [4]–[7].

The fact that racing drivers deliberately drift through sharp corners indicates that there is a lot of knowledge about agile control to be learned. During drifting, a series of high-frequency decisions like steering and throttle should be executed precisely and safely. Therefore, by studying drift behaviors, we can design controllers which fully exploit vehicle dynamics to reduce lap time with high-speed sideslip cornering for racing games. The results could further contribute to the understanding of aggressive driving techniques and extend the operating envelope for autonomous vehicles.



(a) Drifting through a corner (b) Normal driving through a corner

Fig. 1. Comparison between drifting and normal driving through a corner. A drift car usually has a large slip angle β with saturated rear tires caused by oversteering, which is often evidenced by large amounts of tire smoke.

Most of the previous works on drift control are based on the understanding of vehicle dynamics [8]–[12], including tire forces and moments generated by the wheels. Then models with varying fidelities and explicit motion equations are utilized to develop the required controllers by classical, adaptive or optimal control methods. However, in these methods, tire parameters such as longitudinal stiffness at different operating points have to be identified in advance, which is extremely complicated and costly [3]. It is also not easy to accurately derive the entire vehicle dynamics, because some parts of the system are hard to model, and exceeding the handling limits of these models could lead to strong input coupling and sideslip instability [8].

The aforementioned limitations motivate the exploration of strategies to agilely control the drifting vehicles without tire models or explicit motion equations. It is a perfect use case for the learning-based methods, especially model-free deep reinforcement learning (RL). Instead of relying on the human understanding of the world to design controllers, model-free deep RL methods learn the optimal policy by *interacting* with the environment. Prior learning-based works on autonomous drifting [6], [13] mostly consider *sustained* drift by stabilizing the vehicle states about a single drift equilibrium (e.g., steady state circular drift), which is straightforward but not practical. Thus, a novel learning-based method to realize high-speed *transient* drift by tracking a set of non-steady drift states (e.g., drift cornering) is discussed in this paper. The main contributions of this paper are as follows.

- We design a closed-loop controller based on model-free deep RL to control front-wheel drive (FWD) vehicles to drive at high speed (80-128 km/h), and to drift through sharp corners quickly and stably following a reference trajectory, as shown in Fig. 2.

*The first two authors contributed equally to this work.

[1]P. Cai, X. Mei, Y. Sun and M. Liu are with The Hong Kong University of Science and Technology (email: pcaiaa, xmeiab, eeyxsun, eelium@ust.hk).

[2]L. Tai is with Alibaba Group (email: tailei.tl@alibaba-inc.com).

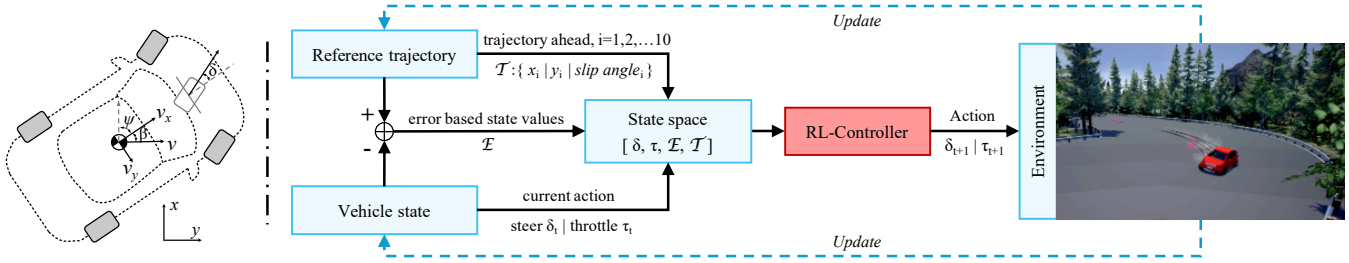


Fig. 2. State variables of the vehicle (left) and the control loop of our deep RL-based method (right). The steering angle is δ. The heading angle ψ is defined as the angle between the direction of the heading and the direction of the world-frame x. The forward and side velocities of the vehicle are v_x and v_y respectively, with v being the total velocity. The angle between the direction of the heading and the direction of v is called the slip angle β. For the control loop, the deep RL-Controller receives observations from the neighboring reference trajectory and the vehicle state. Then it produces an action composed of the steering angle and throttle to operate the simulated car. Finally, the environment feeds back the updated vehicle state and the reference trajectory, to be utilized in the next control step.

- We evaluate the proposed controller on various environmental configurations (corner shapes, vehicle types/mass and tire friction) and show its notable generalization ability.
- We open source our code for benchmark tests and present a dataset for future studies on autonomous drifting. The dataset contains seven racing maps with reference drift trajectories. [1]

II. Related Work

A. Reinforcement learning algorithms

Reinforcement learning is an area of machine learning concerning how agents should take actions to maximize the sum of expected future rewards. The action ($\mathbf{a}_t$) is taken according to a policy $\pi : \mathbf{s}_t \rightarrow \mathbf{a}_t$, where $\mathbf{s}_t$ is the current state. The policy is then evaluated and updated through repeated interactions with the environment by observing the next state ($\mathbf{s}_{t+1}$) and the received reward (r_t).

RL algorithms are divided into model-based and model-free types. Different from model-based RL algorithms such as probabilistic inference for learning control (PILCO) [14], model-free RL eliminates the complex and costly modeling process entirely. Combined with deep neural networks as nonlinear function approximators, model-free RL has been applied to various challenging areas. The algorithms can be divided into value-based and policy gradient algorithms. Value-based methods, such as DQN [15], learn the state (or action) value function and select the best action from a discrete space, while policy gradient methods directly learn the optimal policy, which extend to a continuous action space. The actor-critic framework is widely used in policy gradient methods. Based on this framework, Lillicrap et al. [16] propose deep deterministic policy gradients (DDPG) with an off-policy learning strategy, where the previous experience can be used with a memory replay buffer for better sample efficiency. However, this method is difficult to converge due to the limited exploration ability caused by its deterministic character. To improve the convergence ability and avoid the high sample complexity, one of the leading state-of-the-art methods called soft actor-critic (SAC) [17] is proposed. It learns a stochastic actor with an off-policy strategy, which ensures sufficient exploration and efficiency for complex tasks.

[1]https://sites.google.com/view/autonomous-drifting-with-drl/

B. Drifting control approaches

1) Traditional methods: Different levels of model fidelity depicting the vehicle dynamics have been used in prior works for the drift controller design. A two-state single-track model is used by Voser et al. [2] to understand and control high sideslip drift maneuvers of road vehicles. Zubov et al. [1] apply a more-refined three-state single-track model with tire parameters to realize a controller stabilizing the all-wheel drive (AWD) car around an equilibrium state in the Speed Dreams Simulator.

Although these methods have been proposed to realize steady-state drift, transient drift is still an open problem for model-based methods, mainly due to the complex dynamics while drifting. Velenis et al. [11] introduce a bicycle model with suspension dynamics and apply different optimization cost functions to investigate drift cornering behaviors, which is validated in the simulation. For more complex trajectories, Goh et al. [8] use the rotation rate for tracking the path and yaw acceleration for stabilizing the sideslip, and realize automated drifting along an 8-shaped trajectory.

These traditional drift control methods rely on the knowledge of tire or road forces, which cannot be known precisely due to the real-world environmental complexity. In addition, inaccuracies in these parameters will lead to poor control performance.

2) Learning-based methods: Cutler et al. [13] introduce a framework that combines simple and complex simulators with a real-world remote-controlled car to realize a steady-state drift with constant sideways velocity, in which a model-based RL algorithm, PILCO, is adopted. Bhattacharjee et al. [6] also utilize PILCO to realize sustained drift for a simple car in the Gazebo simulator. Acosta et al. [3] propose a hybrid structure formed by the model predictive controller (MPC) and neural networks (NNs) to achieve drifting along a wide range of road radii and slip angles in the simulation. The NNs are used to provide reference parameters (e.g., tire parameters) to the MPC, which are trained via supervised learning.

Our work differs from the aforementioned works in several major ways. First, we adopt SAC, the state-of-the-art model-free deep RL algorithm, to train a closed-loop drift controller.

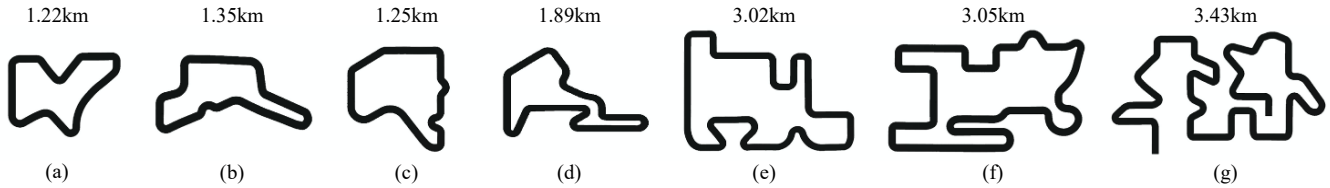


Fig. 3. Seven maps are designed for the drifting task. The difficulty of driving increases from (a) to (g). (a-f) are for training and (g) is for evaluation.

To the best of the authors knowledge, this is the first work to achieve transient drift with deep RL. Second, our drift controller generalizes well on various road structures, tire friction and vehicle types, which are key factors for controller design but have been neglected by prior works in this field.

III. Methodology

A. Formulation

We formulate the drift control problem as a trajectory following task. The goal is to control the vehicle to follow a trajectory at high speed (>80 km/h) and drift through manifold corners with large sideslip angles (>20°), like a professional racing driver. We design our controller with SAC and use CARLA [18] for training and validation. CARLA is an open-source simulator providing a high-fidelity dynamic world and different vehicles of realistic physics.

1) Map generation: Seven maps (Fig. 3) with various levels of difficulty are designed for the drifting task, for which we refer to the tracks of a racing game named PopKart [19]. These are generated by RoadRunner [20], a road and environment creation software for automotive simulation.

2) Trajectory generation: For a specific environment, we aim to provide our drift controller with a candidate trajectory to follow. However, the prior works from which to generate reference drift trajectories [8], [9] are based on simplified vehicle models, which are rough approximations of the real physics. To better train and evaluate our controller, more suitable trajectories are needed. To this end, we invite an experienced driver to operate the car with steering wheel and pedals (Logitech G920) on different maps and record the corresponding trajectories. The principle is to drive as fast as possible and use drift techniques for cornering sharp bends. The collected data contains the vehicle world location, heading angles, body-frame velocities and slip angles, to provide reference states for training and evaluation.

B. RL-based drift controller

1) State variables: The state variables of the vehicle include steering angle δ, throttle τ, forward and side velocities (v_x, v_y), total velocity v, side slip angle β and heading angle ψ, as depicted in Fig. 2. For an arbitrary location of the vehicle, we adopt the vector field guidance (VFG) [21] to determine the desired heading angle ψ^d. Fig. 4 demonstrates a VFG for a linear path and related error variables, which are cross track error e_y and heading angle error e_ψ. The objective of the constructed vector field is that when e_y is small, ψ^d is close to

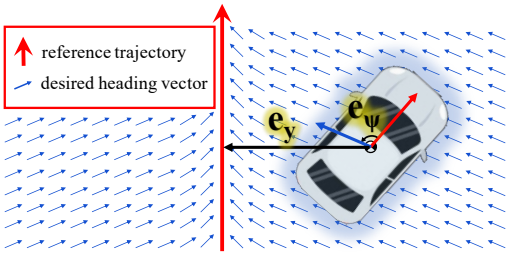


Fig. 4. Vector field guidance (VFG) for drift control. e_y is the cross track error, defined as the perpendicular distance of the vehicle from the reference track. e_ψ is the heading angle error, which is the difference between the heading angle of the vehicle and the desired heading angle provided by VFG.

the direction of the reference trajectory ψ^{ref}. As e_y increases, their difference increases as well:

$$\psi^d = d\psi^\infty \frac{2}{\pi}\tan^{-1}(ke_y) + \psi^{ref}, \tag{1}$$

where $d = 1$ if the vehicle is on the west of the reference path, or else $d = -1$. k is a positive constant that influences the rate of the transition from $\left(\psi^{ref} \pm \psi^\infty\right)$ to ψ^{ref}. Large values of k result in short and abrupt transitions, while small values cause long and smooth transitions. In this work, we choose $k = 0.1$. ψ^∞ is the maximum deviation between ψ^d and ψ^{ref}, which is set to 90°.

2) State space: Based on the state variables introduced above, the state space $\mathbf{s} \in \mathcal{S}$ is defined as (2),

$$\mathcal{S} = \left\{\delta, \tau, e_y, \dot{e}_y, e_\psi, \dot{e}_\psi, e_\beta, \dot{e}_\beta, e_{vx}, \dot{e}_{vx}, e_{vy}, \dot{e}_{vy}, \mathcal{T}\right\}, \tag{2}$$

where $\mathcal{T}$ contains ten (x, y) positions and slip angles in the reference trajectory ahead. Therefore, the dimension of $\mathcal{S}$ is 42. e_β is the slip angle difference between the vehicle and the reference trajectory. e_{vx} and e_{vy} is the error of the forward and side velocity, respectively. Moreover, time derivatives of the error variables, such as $\dot{e}_y$, are included to provide temporal information to the controller [22]. We also define the terminal state with an *endFlag*. When the vehicle is in collision with barriers, arrives at the destination or is over fifteen meters away from the track, *endFlag* becomes *true* and the current state changes to terminal state $\mathbf{s}_T$.

3) Action space: The continuous action space $\mathbf{a} \in \mathcal{A}$ is defined as (3),

$$\mathcal{A} = \{\delta, \tau\}. \tag{3}$$

In CARLA, the steering angle δ and throttle τ is normalized to $[-1, 1]$ and $[0, 1]$, respectively. Since the vehicle is expected to drive at high speed, we further limit the range of the throttle to $[0.6, 1]$ to prevent slow driving and improve training efficiency. Additionally, according to the control test in CARLA, high-speed vehicles are prone to rollover if large

Algorithm 1: SAC controller training algorithm

Data: Buffer $\mathcal{D}$, total number of transitions N, update threshold η, number of updates λ
Result: Optimal control policy π_ϕ^*

Initialize parameters of all networks.
$\mathcal{D} \leftarrow \emptyset,\ N \leftarrow 0,\ \mathbf{s}_t \leftarrow \mathbf{s}_0$;
for *each episode* **do**
  while $s_t \neq s_T$ **do**
    $\mathbf{a}_t \sim \pi_\phi(\mathbf{a}_t|\mathbf{s}_t),\ \mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1}|\mathbf{s}_t,\mathbf{a}_t)$;
    $\mathcal{D} \leftarrow \mathcal{D} \bigcup \{(\mathbf{s}_t,\mathbf{a}_t,r(\mathbf{s}_t,\mathbf{a}_t),\mathbf{s}_{t+1})\}$;
    $N \leftarrow N+1,\ \mathbf{s}_t \leftarrow \mathbf{s}_{t+1}$;
  end
  if $N \geqslant \eta$ **then**
    Update all networks λ times;
  end
end

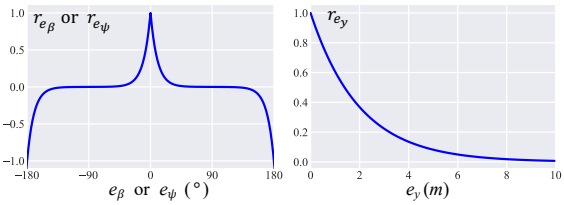


Fig. 5. The partial rewards designed for vehicle drift control. The rewards reach a maximum value when the corresponding error is equal to 0, and decrease as the error increases. When the course angle error e_ψ is larger than 90°, r_{e_ψ} become negative to further indicate a bad control command and prevent the vehicle from driving in the opposite direction.

steering angles are applied. Therefore, the steering is limited to a smaller range of $[-0.8, 0.8]$ to prevent rollover.

Perot et al. [23] successfully used RL to control a simulated racing car. However, we observe a shaky control output in their demonstrated video. To avoid this phenomenon, we impose continuity in the action ($\mathbf{a}_t$), by constraining the change of output with the deployed action in the previous step ($\mathbf{a}_{t-1}$). The action smoothing strategy is

$$\mathbf{a}_t = K_1 \mathbf{a}_t^{net} + K_2 \mathbf{a}_{t-1}, \tag{4}$$

where $\mathbf{a}_t^{net}$ is the action predicted by the network with state $\mathbf{s}_t$. K_1 and K_2 are the tuning diagonal matrices to adjust the smoothing effect. The larger the value of K_2, the more similar $\mathbf{a}_t$ and $\mathbf{a}_{t-1}$, and the smoother the corresponding control effect. Note that $K_{i(11)}$ influences the steering angle and $K_{i(22)}$ influences the throttle. We empirically select the value of $[K_{i(11)}, K_{i(22)}]$ from a range of $\{[0.1,0.9],[0.3,0.7],[0.5,0.5],[0.7,0.3],[0.9,0.1]\}$, and finally set K_1, K_2 as follows.

$$K_1 = \begin{bmatrix} 0.1 & 0 \\ 0 & 0.3 \end{bmatrix}, \quad K_2 = \begin{bmatrix} 0.9 & 0 \\ 0 & 0.7 \end{bmatrix}. \tag{5}$$

4) Reward shaping: A reward function should be well defined to evaluate the controller performance, based on the goal of high-speed drifting through corners with low related errors (e_y, e_ψ, e_β). Therefore, we first design some partial

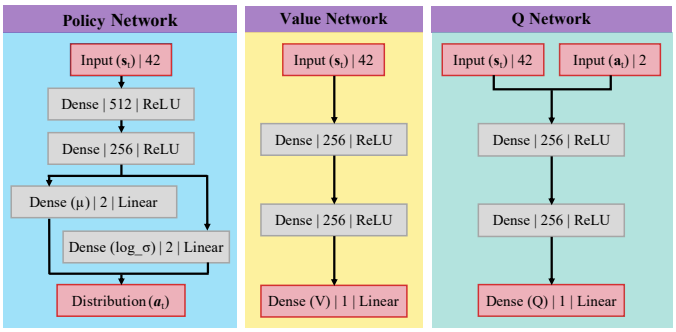


Fig. 6. SAC network structures. The instructions in every layer indicate the network layer type, output channel dimension and activation function. *Linear* here means no activation functions are used and *Dense* means a fully connected neural network.

rewards $r_{e_y}, r_{e_\psi}, r_{e_\beta}$ as (6), and illustrate them in Fig. 5.

$$r_{e_y} = e^{-k_1 e_y}$$
$$r_{e_\psi},\ r_{e_\beta} = f(x) = \begin{cases} e^{-k_2|x|} & |x| < 90° \\ -e^{-k_2(180°-x)} & x \geq 90° \\ -e^{-k_2(180°+x)} & x \leq -90° \end{cases} \tag{6}$$

Note that r_{e_ψ} and r_{e_β} have the same computational formulae, which is denoted as $f(x)$, with x representing e_ψ or e_β. k_1 and k_2 are selected as 0.5 and 0.1. The total reward is defined as (7), which is the product of the vehicle speed and the weighted sum of partial rewards:

$$r = v(k_{e_y} r_{e_y} + k_{e_\psi} r_{e_\psi} + k_{e_\beta} r_{e_\beta}). \tag{7}$$

Speed factor v is used to stimulate the vehicle to drive fast. If v is smaller than 6 m/s, the total reward is decreased by half as a punishment; otherwise, the reward is the original product. The weight variables $[k_{e_y}, k_{e_\psi}, k_{e_\beta}]$ are set to [40,40,20]. We empirically select these values from a range of $\{[4,4,2],[20,20,20],[40,40,20],[400,400,200]\}$.

5) Soft actor-critic: We choose SAC as our training algorithm, which optimizes a stochastic policy by maximizing the trade-off between the expected return and entropy with the off-policy learning method. It is based on the actor-critic framework, where the policy network is the actor, and the Q-network together with the value network is the critic. The critic can suggest a convergence direction for the actor to learn the optimal policy. In our experiments, three kinds of networks, including the policy network (π_ϕ), value network (V_ψ) and Q-networks (Q_{θ_1}, Q_{θ_2}) are learned. The different network structures are presented in Fig. 6. In particular, two Q-networks with the same architecture are trained independently as the clipped double-Q trick, which can speed up training in this hard task, and the value network is used to stabilize the training. For more detailed information about the algorithm, we refer the reader to [17].

The complete training algorithm is shown in Algorithm 1. Firstly, the agent observes the current 42-dimensional state $\mathbf{s}_t$, which is then transferred to a 2-dimensional action $\mathbf{a}_t$ with fully-connected layers by the policy network. The action is sampled from the output distribution and normalized to $[-1, 1]$ with the *tanh* activation function. The sampled action is further mapped and smoothed to interact with the environment. When

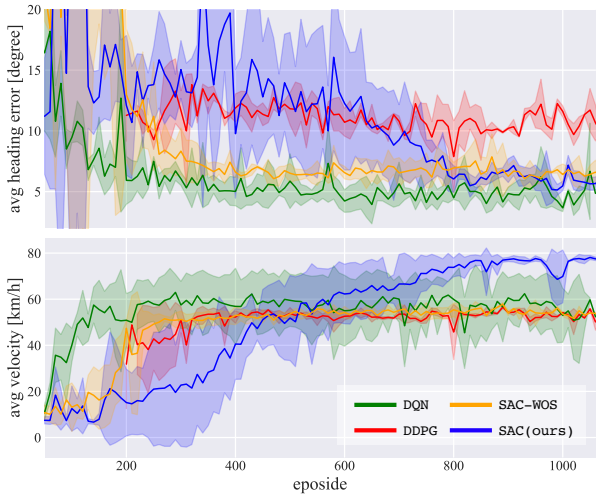


Fig. 7. Performance curves of different algorithms during training on map (a). The plots are averaged over 3 repeated experiments. The solid curve corresponds to the mean, and the shaded region to the standard deviation. Note that the DDPG controller starts to be evaluated from the 200th episode, because the vehicle often gets stuck in circling around the start location in the early phase.

the agent obtains the next state $\mathbf{s}_{t+1}$ and reward $r(\mathbf{s}_t, \mathbf{a}_t)$, the transition $(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})$ is stored into the replay buffer. Such interaction and stored procedures are repeated during training. At the end of the episodes, when the number of transitions is larger than the setting threshold, networks are updated respectively with the functions $J_V(\psi)$, $J_Q(\theta_1)$, $J_Q(\theta_2)$ and $J_\pi(\phi)$, which are the same as those defined in [17]. The whole procedure is repeated until the optimal policy is learned.

IV. Experiments and Discussion

A. Training setup

1) Implementation: We train our SAC controller on six maps (Fig. 3 (a-f)). Map (a) is relatively simple and is used for the first-stage training, in which the vehicle learns some basic driving skills such as speeding up by applying large values of throttle and drifting through some simple corners. Maps (b-f) have different levels of difficulty with diverse corner shapes, which are used for further training with the pre-trained weights from map (a). The vehicle can use the knowledge learned from map (a) and quickly adapt to these tougher maps, to learn a more advanced drift technique. In this stage, maps (b-f) are randomly chosen for each training episode. In addition to the various road structures, we also hope the controller can handle other changing conditions. To this end, at the start of each episode, the tire friction and vehicle mass is sampled from the range of $[3.0, 4.0]$ and $[1.7t, 1.9t]$ respectively. Lower values make the vehicle more prone to slip, thus leading to a harder control experience. We use the Adam optimizer for training with a learning rate of 0.0003 and batch size of 512.

2) Baselines: For comparison, we train the controller with three other methods:

- DQN. Since it can only handle the discrete action space, we divide the range of the steering angle evenly for 10 values and throttle for 5 values. Thus, the number of candidate actions is 50 without the action smoothing strategy.
- DDPG. For better performance of this method, we set $K_{1(11)} = 0.6$ and $K_{2(11)} = 0.4$ in (5).
- SAC-WOS. We use SAC to train the controller but without the action smoothing strategy.

3) Performance during training: Fig. 7 shows the average heading angle error and the average speed of evaluation rollouts during training for DQN, DDPG, SAC-WOS and SAC. The results show that all methods can learn to speed up and reduce the error during training, and finally converge to their optimal values. In the end, they have approximately the same heading angle error, except for DDPG. However, SAC achieves a much higher average velocity (80 km/h) than the baselines. This illustrates that the SAC controller is capable of making the vehicle follow the reference trajectory accurately as well as maintain a high speed. In addition, it is shown that the action smoothing strategy can improve the final performance by comparing SAC-WOS and SAC.

B. Evaluation

To evaluate the controller performance, we select three combinations of tire friction (F) and vehicle mass (M) as F3.0M1.7, F3.5M1.8 and F4.0M1.9. The test environment is an unseen tough map (g) with various corners of angles ranging from 40° to 180°.

1) Performance metrics: We adopt seven metrics to measure the performance of different methods.

- **C.T.E.** and **H.A.E.** is the cross track error and heading angle error, respectively.
- **MAX-VEL** and **AVG-VEL** is the maximum and average velocity of a driving test, respectively.
- **L.T.** is the time to reach the destinations (the lap time).
- **SMOS** measures the smoothness of driving, calculated by the rolling standard deviation of steering angles during a driving test.
- **SLIP** is the maximum slip angle during a driving test. Since larger slip angles mean larger usable state spaces beyond the handling limits, it can indicate a more powerful drift controller.

2) Quantitative results: All controllers are tested four times on map (g) and the average evaluation results are presented in Table I. Apart from the overall performance through the whole track, the results for driving through corners are also listed, to give a separate analysis on drift ability. Additionally, two reference results on F3.5M1.8 from the human driver are presented for comparison, in which HUMAN-DFT drifts through sharp corners and HUMAN-NORM slows down and drives cautiously through corners.

Time cost and velocity. Our SAC controller achieves the shortest lap time in all setups, with the maximum velocity among the four methods. In particular, the speed reaches up to 103.71 km/h in setup F3.0M1.7, which is much higher than DDPG (90.59 km/h) and SAC-WOS (84.02 km/h). Compared with HUMAN-NORM, the SAC controller adopts the drifting strategy, which achieves a much shorter lap time.

Error analysis. C.T.E. and H.A.E. indicate whether the vehicle can follow the reference trajectory accurately. The SAC-WOS controller achieves the smallest C.T.E., but SAC is

TABLE I
QUANTITATIVE EVALUATION AND GENERALIZATION FOR DIFFERENT METHODS UNDER VARIED ENVIRONMENT SETUPS. ↑ MEANS LARGER NUMBERS ARE BETTER, ↓ MEANS SMALLER NUMBERS ARE BETTER. THE BOLD FONT HIGHLIGHTS THE BEST RESULTS IN EACH COLUMN.

	Setup	Methods	Performance over the whole track					Performance through corners			
			C.T.E.↓ (m)	H.A.E.↓ $(^\circ)$	MAX-VEL↑ (km/h)	L.T.↓ (s)	SMOS↓	C.T.E.↓ (m)	H.A.E.↓ $(^\circ)$	AVG-VEL↑ (km/h)	SLIP↑ $(^\circ)$
Evaluation	F3.0M1.7	DQN	1.286	7.971	98.57	150.89	0.132	1.413	3.127	77.49	19.48
		DDPG	2.036	16.664	90.59	215.45	0.401	2.674	4.106	52.52	29.01
		SAC-WOS	**0.811**	6.703	84.02	186.12	0.632	**1.030**	4.559	60.82	23.84
		SAC	0.900	**5.707**	**103.71**	**145.90**	**0.130**	1.174	**3.009**	**78.85**	**29.23**
	F3.5M1.8	DQN	1.125	7.118	96.45	149.34	0.131	1.277	3.957	76.85	24.12
		DDPG	1.805	16.921	84.27	216.25	0.402	2.178	4.556	52.21	25.28
		SAC-WOS	**0.734**	6.434	84.04	187.57	0.642	**0.970**	4.342	60.49	24.76
		SAC	0.907	**5.776**	**103.45**	**143.42**	**0.125**	1.361	**2.129**	**79.07**	**26.17**
	F4.0M1.9	DQN	1.114	6.943	96.97	149.08	0.131	1.330	3.918	76.74	20.76
		DDPG	1.629	15.899	82.59	212.94	0.402	1.897	4.005	52.94	21.62
		SAC-WOS	**0.736**	6.170	81.12	191.25	0.655	**1.006**	4.064	59.07	23.85
		SAC	0.920	**5.850**	**102.78**	**142.44**	**0.123**	1.526	**1.691**	**78.93**	**24.28**
Generalization	F2.6M1.6	SAC	1.219	6.757	105.69	148.87	0.132	1.346	3.443	79.46	43.27
	F4.4M2.0		1.001	6.351	100.83	143.21	0.124	1.701	1.477	78.20	23.08
	DF-M1.8		1.021	6.718	102.46	144.70	0.129	1.186	2.748	80.13	12.54
	Vehicle-2		1.000	6.252	128.07	126.28	0.140	1.345	1.382	92.77	45.03
	Vehicle-3		0.918	6.368	123.75	126.55	0.138	1.322	1.687	91.23	39.38
	Vehicle-4		0.450	3.486	67.46	187.94	0.103	0.659	1.514	59.46	9.74
AppTest	F3.5M1.8	HUMAN-DFT	—	—	112.86	141.69	0.055	—	—	79.38	28.59
		HUMAN-NORM	—	—	108.58	160.53	0.011	—	—	66.57	8.28
		SAC-APP	—	—	105.59	143.59	0.106	—	—	80.30	26.87

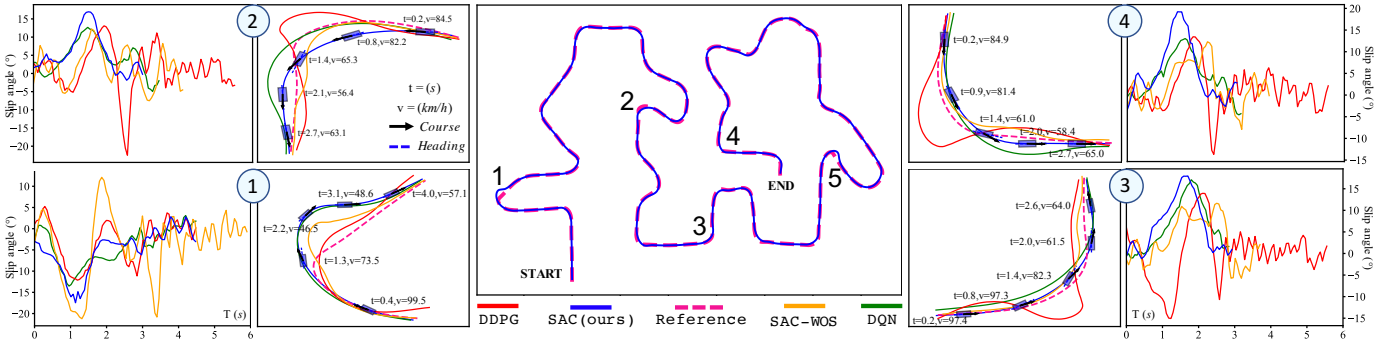


Fig. 8. Qualitative trajectory results on map (g) based on the setup F3.5M1.8. The picture in the middle represents the overall trajectory of the human driver (i.e., Reference) and our SAC controller. The pictures on either side depict some drift-cornering trajectories and corresponding slip angle curves from different controllers. For further analysis of the SAC controller, we label some state information over time along these trajectories, which are velocity, moving direction (course) and heading direction. Note that the difference between the course and the heading is the slip angle.

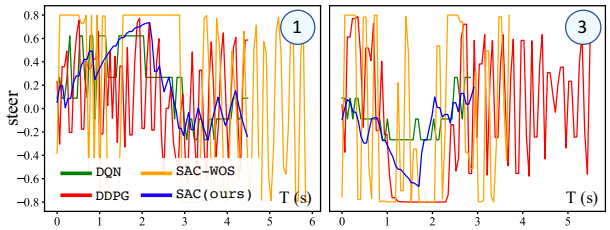


Fig. 9. The curves of steering control command through corner-1 and corner-3 from different controllers.

the best for H.A.E., especially through corners. The possible reason is SAC controls the car to drift through corners with similar slip angles to the reference behaviors, while other methods tend to mainly track the positions on the trajectory.

Drifting velocity and slip angle. We calculate the average velocity and the largest slip angle while drifting. In all setups, the SAC controller achieves the highest speed and largest slip angles. In setup F3.5M1.8, the AVG-VEL reaches up to 79.07 km/h, which is very similar to HUMAN-DFT (79.38 km/h). In setup F3.0M1.7, the SLIP of the SAC controller reaches up to 29.23°, which is much higher than DQN and SAC-WOS. On the other hand, although the DDPG and SAC-WOS controller can generate large slip angles, their control outputs are rather shaky, leading to velocities even lower than HUMAN-NORM.

Driving smoothness. SMOS reflects how steady the vehicle is while driving. Although all controllers generate larger values of SMOS than the human driver, SAC achieves the smallest among them.

TABLE II
VEHICLES USED FOR TRAINING AND TESTING OUR MODEL. THE VEHICLE USED FOR TRAINING IS BOLDFACED. MOI IS THE MOMENT OF INERTIA OF THE ENGINE AROUND THE AXIS OF ROTATION.

Physics	**Vehicle-1** ***(Audi A2)***	Vehicle-2 *(Audi TT)*	Vehicle-3 *(Citroen C3)*	Vehicle-4 *(ColaTruck)*
Mass (t)	1.7-1.9	1.2	1.0	5.5
Tire friction	3-4	3.5	3.5	3.5
MOI (kgm^2)	0.1	0.3	1.0	1.0
Length (m)	3.6	4.2	3.8	5.6
Width (m)	1.6	1.8	1.7	2.5
Drive type	FWD	FWD	FWD	AWD

Fig. 10. Vehicles used for training and testing our model.

3) Qualitative results: Fig. 8 shows the qualitative trajectory results on the test map (g). The SAC controller is shown to have excellent performance in tracking the trajectory on linear paths and most of the corners. Some mismatches may occur if the corner angle is too small (e.g., <50°), such as corner-1 and corner-5. In corner-3 and corner-4 with angles of about 90°, the drift trajectory of our SAC controller is very similar to that of the reference, even though the speed at the entry of corner-3 is near 100 km/h. During the drifting process of SAC, the speed is reduced when the vehicle drives close to the corner vertex, resulting from the large slip angle. However, it still maintains a high speed and accelerates quickly when leaving the corner.

Since all controllers output the largest value of throttle to maintain a high speed, except DQN (which jumps between 0.9 and 1.0), we plot their steering curves for corner-1 and corner-3 for further comparison. These are presented in Fig. 9. It is shown that the steering angles of the other controllers are tremendously shaky, especially for DDPG and SAC-WOS. In contrast, the steering angle of SAC controller concentrates in a smaller range and is much smoother.

C. Generalization

To test the generalization ability of the proposed SAC controller, we evaluate it with varied tire friction, vehicle mass and vehicle types on map (g). Different vehicles and their physics are shown in Fig. 10 and Table II. The performance results are presented in Table I-Generalization.

1) Unseen mass and friction: We set two combinations of unseen friction and mass on vehicle-1 as F2.6M1.6 and F4.4M2.0. Our SAC controller can handle them without any fine-tuning and the highest speed is more than 100 km/h. Drifting is completed successfully and the maximum slip angle is up to 43.27° for F2.6M1.6. Additionally, we test the proposed controller using vehicle-1 with different tire friction in the front wheels (2.8) and rear wheels (4.2). This is called DF-M1.8, since sometimes wear conditions vary on different tires for a vehicle. In this tough setup, our SAC controller can make the vehicle drive through the whole map fast and smoothly. However, the drift control performance does not meet with expectations, with the maximum slip angle smaller than 20°. This is caused by the large rear tire friction, which makes it difficult for the car to slip.

2) Unseen vehicle types: The SAC controller is further tested by three other types of vehicles. Vehicle-2 is similar to Vehicle-1 but is about 0.5t lighter, Vehicle-3 has a much larger MOI and bigger mass, and Vehicle-4 is an all-wheel drive heavy truck with distinct physical parameters. The results show that our SAC method achieves a notable generalization performance on these unseen vehicles. For Vehicle-2, the highest speed is up to 128.07 km/h, and the average drift speed is 92.77 km/h, both of which are even better than Vehicle-1 with the benefit of a smaller mass and a more powerful engine. The same is true of Vehicle-3. For Vehicle-4, the SAC controller is sufficiently capable of controlling it to follow a reference trajectory precisely, but the drift performance is not satisfactory with small slip angles and cornering speeds, due to its heavy weight and large size. Note that for each kind of vehicle, the referenced drift trajectories are different in order to meet the respective physical dynamics.

3) Application test without expert reference: To evaluate whether the proposed SAC model can be deployed in scenarios where expert driving trajectories are not available, we further test it by providing less information. In CARLA, the (x,y) waypoints in the center of the road can easily be obtained, so they are used to form a rough reference trajectory. The directions of this trajectory ψ^{ref} are derived based on its tangents for calculating the heading angle errors e_ψ, and the reference slip angles are set to zero. Accordingly, $e_\beta, \dot{e}_\beta, e_{vy}$ and $\dot{e}_{vy}$ in the state space are also set to zero. The reference forward velocities are set to 110 km/h constant along the whole track. Based on this setup, Vehicle-1 of F3.5M1.8 is tested on map (g) and the results are shown in Table I-AppTest as SAC-APP. It is very interesting that although a rough trajectory is used, the final performance is still comparable with the SAC controller provided with accurate expert trajectories.

Since we mainly exclude the information of slip angle here, it can be inferred that they are dispensable for the policy execution in our task. This phenomenon is valuable, indicating that our drift controller could be applied to unseen tracks without generating an accurate reference trajectory in advance. This is critical for further real-world applications where a rough reference could be derived online from 2D or 3D maps, which are common in robot applications.

D. Ablation study

We have shown above that a rough trajectory is sufficient for the application. Therefore, can we also provide less in-

TABLE III
QUANTITATIVE EVALUATION FOR POLICIES TRAINED WITH (SAC-42) OR WITHOUT (SAC-30) SLIP ANGLE AS GUIDANCE. ↑ MEANS LARGER NUMBERS ARE BETTER, ↓ MEANS SMALLER NUMBERS ARE BETTER. THE BOLD FONT HIGHLIGHTS THE BEST RESULTS IN EACH COLUMN.

Setup	Methods	Training	Performance over the whole track					Performance through corners			
		Time ↓ ($hours$)	C.T.E.↓ (m)	H.A.E.↓ ($^\circ$)	MAX-VEL↑ (km/h)	L.T.↓ (s)	SMOS↓	C.T.E. ↓ (m)	H.A.E. ↑ ($^\circ$)	AVG-VEL↑ (km/h)	SLIP↑ ($^\circ$)
F3.5M1.8	SAC-42	**11.03**	**0.896**	**5.390**	**122.44**	**48.01**	**0.094**	**1.298**	**5.992**	86.06	**32.85**
	SAC-30	23.47	1.390	7.908	114.97	51.60	0.106	2.671	15.943	**86.75**	28.47

formation during the training and achieve no degradation in the final performance? To answer this question, a comparison experiment on map (a) is conducted by training an additional controller excluding variables related to the slip angle in the reward and state space ($e_\beta, \dot{e}_\beta$ and 10 reference slip angles in $\mathcal{T}$). In this way the state space becomes 30-dimensional. Accordingly, the corresponding controller is named SAC-30, and SAC-42 indicates the original one. These controllers are tested four times and the average evaluation results are presented in Table III. It shows that SAC-42 costs much less training time but achieves better performance with a higher speed, shorter lap time and smaller error. It also drives more smoothly than SAC-30. Generally, accurate slip angles from expert drift trajectories are indeed necessary in the training stage, which can improve the final performance and the training efficiency.

V. CONCLUSION

In this paper, to realize high-speed drift control through manifold corners for autonomous vehicles, we propose a closed-loop controller based on the model-free deep RL algorithm soft actor-critic (SAC) to control the steering angle and throttle of simulated vehicles. The error-based state and reward are carefully designed and an action smoothing strategy is adopted for stable control outputs. Maps with different levels of driving difficulty are also designed to provide training and testing environments.

After the two-stage training on six different maps, our SAC controller is sufficiently robust against varied vehicle mass and tire friction to drift through complex curved tracks quickly and smoothly. In addition, its remarkable generalization performance has been demonstrated by testing different vehicles with diverse physical properties. Moreover, we have discussed the necessity of slip angle information during training, and the non-degraded performance with a rough and easy-to-access reference trajectory during testing, which is valuable for applications. To reduce the labor costs in generating accurate references for training, we will explore leaning-based methods for trajectory planning in drift scenarios, which is left to future work.

REFERENCES

[1] I. Zubov, I. Afanasyev, A. Gabdullin, R. Mustafin, and I. Shimchik, "Autonomous drifting control in 3d car racing simulator," in *2018 International Conference on Intelligent Systems (IS)*. IEEE, 2018, pp. 235–241.

[2] C. Voser, R. Y. Hindiyeh, and J. C. Gerdes, "Analysis and control of high sideslip manoeuvres," *Vehicle System Dynamics*, vol. 48, no. S1, pp. 317–336, 2010.

[3] M. Acosta and S. Kanarachos, "Teaching a vehicle to autonomously drift: A data-based approach using neural networks," *Knowledge-Based Systems*, vol. 153, pp. 12–28, 2018.

[4] F. Zhang, J. Gonzales, K. Li, and F. Borrelli, "Autonomous drift cornering with mixed open-loop and closed-loop control," *IFAC-PapersOnLine*, vol. 50, no. 1, pp. 1916–1922, 2017.

[5] F. Zhang, J. Gonzales, S. E. Li, F. Borrelli, and K. Li, "Drift control for cornering maneuver of autonomous vehicles," *Mechatronics*, vol. 54, pp. 167–174, 2018.

[6] S. Bhattacharjee, K. D. Kabara, R. Jain, and K. Kabara, "Autonomous drifting rc car with reinforcement learning," 2018.

[7] P. Frère, *Sports Car and Competition Driving*. Pickle Partners Publishing, 2016.

[8] J. Y. Goh, T. Goel, and J. C. Gerdes, "A controller for automated drifting along complex trajectories," in *14th International Symposium on Advanced Vehicle Control (AVEC 2018)*, 2018.

[9] J. Y. Goh and J. C. Gerdes, "Simultaneous stabilization and tracking of basic automobile drifting trajectories," in *2016 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2016, pp. 597–602.

[10] E. Velenis, D. Katzourakis, E. Frazzoli, P. Tsiotras, and R. Happee, "Steady-state drifting stabilization of rwd vehicles," *Control Engineering Practice*, vol. 19, no. 11, pp. 1363–1376, 2011.

[11] E. Velenis and P. Tsiotras, "Minimum time vs maximum exit velocity path optimization during cornering," in *2005 IEEE international symposium on industrial electronics*. Citeseer, 2005, pp. 355–360.

[12] R. Y. Hindiyeh and J. C. Gerdes, "A controller framework for autonomous drifting: Design, stability, and experimental validation," *Journal of Dynamic Systems, Measurement, and Control*, vol. 136, no. 5, p. 051015, 2014.

[13] M. Cutler and J. P. How, "Autonomous drifting using simulation-aided reinforcement learning," in *2016 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2016, pp. 5442–5448.

[14] M. Deisenroth and C. E. Rasmussen, "Pilco: A model-based and data-efficient approach to policy search," in *Proceedings of the 28th International Conference on machine learning (ICML-11)*, 2011, pp. 465–472.

[15] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, p. 529, 2015.

[16] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.

[17] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," *arXiv preprint arXiv:1801.01290*, 2018.

[18] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "Carla: An open urban driving simulator," *arXiv preprint arXiv:1711.03938*, 2017.

[19] Nexon, "Popkart," 2019. [Online]. Available: https://popkart.tiancity.com/homepage/v2/guide/runway.html

[20] VectorZero, "Roadrunner," 2019. [Online]. Available: https://www.vectorzero.io/roadrunner

[21] D. R. Nelson, D. B. Barber, T. W. McLain, and R. W. Beard, "Vector field path following for miniature air vehicles," *IEEE Transactions on Robotics*, vol. 23, no. 3, pp. 519–529, 2007.

[22] J. Woo, C. Yu, and N. Kim, "Deep reinforcement learning-based controller for path following of an unmanned surface vehicle," *Ocean Engineering*, vol. 183, pp. 155–166, 2019.

[23] E. Perot, M. Jaritz, M. Toromanoff, and R. De Charette, "End-to-end driving in a realistic racing game with deep reinforcement learning," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition Workshops*, 2017, pp. 3–4.